



Create Shadow Tables (ORG - USERS) in Your Service

Audience:

backend/integration engineers implementing ORG/USER shadow tables for their service using Key-cloak → RabbitMQ events.

Scope:

- How your service **connects to RabbitMQ** (what you get from the Integration team)
- How to **declare and bind your own queue(s)** to Keycloak exchanges
- The **routing key patterns** for:
 - User events (`user.<plane>.<action>`)
 - Client/org events (`client.<plane>.<action>`)
- Which **binding patterns** to use depending on whether you're a **V1**, **V2**, or “all” consumer
- The **full JSON shapes** of:
 - User messages (create/update/delete)
 - Client / org messages (create/update)
- A **short checklist** to correctly consume and interpret these events

Table Of Content:

[Key-cloak → RabbitMQ — Consumer guide](#)

- [1. What you get from the Integration team](#)
- [2. What you configure in RabbitMQ](#)
- [3. Routing keys \(this is the important part\)](#)
 - [3.1 User events](#)
 - [3.2 Client / organization events](#)
- [4. Which bindings to use](#)
- [5. User message — full JSON shape](#)
 - [5.1 Example — create / update \(all fields\)](#)
 - [5.2 Example — delete \(minimal body\)](#)
- [6. Client / org message — full JSON shape](#)
 - [6.1 Full payload \(all keys\)](#)
- [7. One-page checklist](#)

Key-cloak → RabbitMQ — Consumer guide

You only need to **connect to RabbitMQ**, **bind your queue** to the right **exchange** with a **routing pattern**, then **read the JSON body**. Keycloak already publishes messages; you do not configure Keycloak.

1. What you get from the Integration team

Item	Example / note
Host	e.g. <code>rmq.example.com</code>
Port	<code>5672</code> (AMQP)
Username / password	your service account
Virtual host	often <code>/</code>
User events exchange	default name: <code>keycloak_user_events</code>
Client (org) events exchange	default name: <code>keycloak_client_event</code> <code>s</code>

Both exchanges are **topic** exchanges, **durable**. Each message is **JSON** (`content-type: application/json`), **persistent**.

2. What you configure in RabbitMQ

1. **Declare your own queue** (name you choose, e.g. `Application_name_acme_users`). Use `durable` if you cannot afford to lose undelivered messages when the broker restarts.
2. **Bind** that queue to **each** exchange you care about, using **routing patterns** below.
3. **Do not** create the Keycloak exchanges yourself unless the platform asks you to — usually they already exist. If you declare them, they must be **topic** and **durable**, with the **same names** as above.
4. Make sure you're connected to the dead letter queue.

Typical **consume-only** flow: `connect` → `open channel` → `queue.declare` (your queue) → `queue.bind` (your queue, exchange name, routing pattern) → `basic.consume` .

3. Routing keys (this is the important part)

3.1 User events

Exchange: `keycloak_user_events` (unless the platform gives another name).

Pattern:

```
1 user.<plane>.<action>
2
```

Part	Values	Meaning
plane	v1	Message is meant for v1 consumers.
plane	v2	Message is meant for v2 consumers.
plane	all	Broadcast — meant for everyone who listens to “all”.
action	created , updated , deleted	What happened to the user.

Examples: `user.v1.created` , `user.v2.updated` , `user.all.deleted`

One message has **one** routing key. Broadcast is `user.all.*` , not two copies on `v1` and `v2` .

3.2 Client / organization events

Exchange: `keycloak_client_events` .

Pattern:

```
1 client.<plane>.<action>
2
```

Part	Values	Meaning
plane	v1 / v2 / all	Same idea: targeted v1/v2, or all .
action	created or updated	Organisation (group) was created or

		updated.
--	--	----------

Examples: `client.all.created`, `client.v1.updated`

4. Which bindings to use

Use **topic** binding patterns (`*` = one word, `#` = zero or more words).

If you are...	Bind your queue to...
V1 app — user stream	<code>user.v1.*</code> and <code>user.all.*</code> on <code>keycloak_user_events</code>
V1 app — client stream	<code>client.v1.*</code> and <code>client.all.*</code> on <code>keycloak_client_events</code>
V2 app — user stream	<code>user.v2.*</code> and <code>user.all.*</code> on <code>keycloak_user_events</code>
V2 app — client stream	<code>client.v2.*</code> and <code>client.all.*</code> on <code>keycloak_client_events</code>
One consumer for everything	<code>user.#</code> on user exchange, <code>client.#</code> on client exchange

Why two patterns for V1/V2? Messages sent only

to `user.all.*` do **not** match `user.v1.*`. If you need both “v1-only” and “everyone” traffic, bind `user.v1.*` **and** `user.all.*` (same for clients).

5. User message — full JSON shape

Every user message includes the same keys (missing data is `null`). `event_type` is always one of: `user_created`, `user_updated`, `user_deleted`.

5.1 Example — create / update (all fields)

Routing key can be `user.all.created`, `user.v1.updated`, etc.; the **body** shape is the same.

```

1  {
2    "event_type": "user_created",
3    "keycloak_user_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
4    "id": "10042",
5    "user_uuid": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
6    "username": "jdoe",
7    "first_name": "Jane",
8    "last_name": "Doe",
9    "email": "jane.doe@example.com",
10   "mobile": "+15551234567",
11   "client_id": "org-client-001",
12   "status": "active",
13   "country_code": "US",
14   "dial_code": "+1",
15   "user_type": "participant",
16   "password_set_at": "2026-04-20T08:00:00Z",
17   "preferences": {
18     "locale": "en",
19     "notifications": true
20   },
21   "extra_data": {
22     "department": "Field Ops"
23   },
24   "phone_number_verified_at": "2026-04-21T12:00:00Z",
25   "profile_image": "https://cdn.example.com/avatars/jdoe.png",
26   "created_at": "2026-04-27T09:00:00Z",
27   "updated_at": "2026-04-27T09:30:00Z",
28   "created_by_uuid": "b2c3d4e5-f6a7-8901-bcde-f12345678901",
29   "start_date": "2026-05-01",
30   "end_date": null,
31   "timestamp": "2026-04-27T10:15:30.123456789Z"
32 }
33

```

If `preferences` or `extra_data` is not valid JSON in Keycloak, you receive `null` instead of an object.

5.2 Example — delete (minimal body)

Sometimes only `event_type`, `keycloak_user_id`, and `timestamp` are set; everything else is `null`.

```

1  {
2    "event_type": "user_deleted",
3    "keycloak_user_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
4    "id": null,
5    "user_uuid": null,
6    "username": null,
7    "first_name": null,
8    "last_name": null,
9    "email": null,
10   "mobile": null,
11   "client_id": null,
12   "status": null,
13   "country_code": null,
14   "dial_code": null,
15   "user_type": null,
16   "password_set_at": null,
17   "preferences": null,
18   "extra_data": null,
19   "phone_number_verified_at": null,
20   "profile_image": null,
21   "created_at": null,

```

```

22   "updated_at": null,
23   "created_by_uuid": null,
24   "start_date": null,
25   "end_date": null,
26   "timestamp": "2026-04-27T11:00:05.111222333Z"
27 }
28

```

6. Client / org message — full JSON shape

`event` is always `org_created` or `org_updated`. All other keys are always present in **this order**; unused attributes are `null`. String values usually come from Keycloak group attributes (often `"true"` / `"false"` strings). If a stored value looks like JSON (`{ ... }` or `[ ... ]`) and parses, it is sent as JSON (e.g. `sso_domains` as an array).

6.1 Full payload (all keys)

```

1 {
2   "event": "org_updated",
3   "client_id": "wh-client-acme",
4   "org_key": "acme-field",
5   "org_name": "Acme Field Division",
6   "otp": "true",
7   "subscription_start": "2026-01-01",
8   "subscription_end": "2027-01-01",
9   "timestamp": "2026-04-27T12:00:00.000+00:00",
10  "secondary_name": null,
11  "number_of_admin": "5",
12  "number_of_participant": "120",
13  "number_of_projects": "8",
14  "documenters_count": "15",
15  "balance": "1000.50",
16  "balance_of_users": "800.00",
17  "subscription_type": "enterprise",
18  "contract_reference": "CTR-2026-0042",
19  "logo": "https://cdn.example.com/orgs/acme/logo.svg",
20  "logo_printing": null,
21  "logo_icon": null,
22  "logo_ar": null,
23  "contact_name": "Ops Manager",
24  "contact_email": "ops@acme.example",
25  "contact_phone_number": "+442071234567",
26  "deactivate_at": null,
27  "unifier_token_expires_at": null,
28  "region": "EU",
29  "personalization_flag": "1",
30  "primary_color": "#0B5FFF",
31  "accent_color": "#FF6B00",
32  "secondary_color": "#333333",
33  "dark_mode": "false",
34  "platform_access_mode": "V2_ONLY",
35  "is_sync_enabled": "1",
36  "is_new_client": "false",
37  "placeholder_user_id": null,
38  "enable_bim": "true",
39  "enable_blur_faces": "false",
40  "enable_measurements": "true",
41  "enable_api_access_token": null,
42  "enable_unselected_points_video_report": null,
43  "enable_project_unifier": null,
44  "enable_dms_button": null,

```

```

45  "enable_command_screen_mode": null,
46  "pledge_message_enabled": null,
47  "is_video_report_feature_enabled": null,
48  "custom_fields_enabled": null,
49  "unifier_project_adapter": null,
50  "unifier_client_id": null,
51  "unifier_client_secret": null,
52  "unifier_access_token": null,
53  "enable_p6_unifier_sync": null,
54  "enable_observation_unifier_sync": null,
55  "sso_auth_mode": "OIDC",
56  "sso_domains": ["https://app.acme.example",
57                  "https://admin.acme.example"],
58  "oidc_client_id": "acme-oidc-client",
59  "oidc_client_secret": null,
60  "oidc_scope": "openid profile email",
61  "oidc_discovery_endpoint": "https://idp.example.com/.well-
known/openid-configuration",
62  "keycloak_sso_configuration_status": "configured",
63  "keycloak_sso_configuration_error_message": null,
64  "keycloak_sso_configuration_error_details": null,
65  "keycloak_realm": "acme",
66  "project_structure_enabled": "true",
67  "analytics_enabled": "false",
68  "menu_planning_enabled": null,
69  "planning_gis_enabled": null,
70  "menu_design_enabled": null,
71  "design_ai_compliance_analysis_enabled": null,
72  "design_ai_design_analysis_enabled": null,
73  "design_management_enabled": null,
74  "menu_construction_enabled": null,
75  "construction_gis_enabled": null,
76  "construction_capture_enabled": null,
77  "construction_streaming_enabled": null,
78  "construction_incident_management_enabled": null,
79  "construction_quality_management_enabled": null,
80  "construction_observations_enabled": null,
81  "construction_qhse_enabled": null,
82  "construction_audit_management_enabled": null,
83  "construction_real_time_monitoring_enabled": null,
84  "menu_operations_enabled": null,
85  "operations_ai_visual_pollution_enabled": null,
86  "operations_facility_management_enabled": null,
87  "menu_project_controls_enabled": null,
88  "project_controls_ai_bid_analysis_enabled": null,
89  "project_controls_ccm_enabled": null,
90  "project_controls_dms_enabled": null,
91  "project_controls_schedule_enabled": null,
92  "geospatial_data_enabled": null,
93  "iot_data_enabled": null,
94  "reporting_data_enabled": null,
95  "created_at": "2026-01-15T10:00:00Z",
96  "updated_at": "2026-04-27T12:00:00Z",
97  "created_by": "admin-user-uuid"
98  }

```

In production, any field you do not use may still be `null` or a string; your parser should accept that.

7. One-page checklist

1. Get **host, port, vhost, user, password**, and **exact exchange names** from the integration team.
2. Create **your queue**; bind with the patterns in **\$4**.
3. Parse JSON; use `event_type` (users) or `event` (clients) to know what happened.
4. Optionally log the AMQP **routing key** on each message for support.